

LAB_02_Shiguihara

Intalação de Pacotes

```
# 1. Instalação de todos os pacotes do CRAN
install.packages(c( "ggplot", "dplyr","readr","devtools", "remotes"))
```

Installing packages into '/cloud/lib/x86_64-pc-linux-gnu-library/4.6'
(as 'lib' is unspecified)

Warning: package 'ggplot' is not available for this version of R

A version of this package for your version of R might be available elsewhere,
see the ideas at
<https://cran.r-project.org/doc/manuals/r-patched/R-admin.html#Installing-packages>

```
# 2. Instalação do pacote diretamente do GitHub
remotes::install_github("jayjacobs/ggcal")
```

Skipping install of 'ggcal' from a github remote, the SHA1 (ab1a85ad) has not changed since 1
Use `force = TRUE` to force installation

1. Quais são as estatísticas suficientes para a determinação do percentual de vôos atrasados na chegada ($ARRIVAL_DELAY > 10$)? As Estatística Suficientes para a determinação do percentual de vôos atrasados na chegada são as somatórias de vôos em que houve o atraso de mais de 10 minutos. E a contagem total de voos.
2. Crie uma função chamada `getStats` que, para um conjunto de qualquer tamanho de dados provenientes de `flights.csv.zip`, execute as seguintes tarefas (usando apenas verbos do `dplyr`):

- Filtre o conjunto de dados de forma que contenha apenas observações das seguintes Cias. Aéreas: AA, DL, UA e US;
- Remova observações que tenham valores faltantes em campos de interesse;
- Agrupe o conjunto de dados resultante de acordo com: dia, mês e cia. aérea;
- Para cada grupo em b., determine as estatísticas suficientes apontadas no item 1. e os retorne como um objeto da classe tibble;
- A função deve receber apenas dois argumentos:
 - `input`: o conjunto de dados (referente ao lote em questão);
 - `pos`: argumento de posicionamento de ponteiro dentro da base de dados. Apesar de existir na função, este argumento não será empregado internamente. É importante observar que, sem a definição deste argumento, será impossível fazer a leitura por partes.

```
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

`filter`, `lag`

The following objects are masked from 'package:base':

`intersect`, `setdiff`, `setequal`, `union`

```
getStats <- function(input, pos) {
  resultado <- input |>
  filter(AIRLINE %in% c("AA", "DL", "UA", "US"), !is.na(ARRIVAL_DELAY)) |>
  group_by(MONTH, DAY, AIRLINE) |>
  summarise(
    voos_atrasados = sum(ARRIVAL_DELAY > 10),
    total_voos = n(),
    .groups = "drop"
  )

  return(resultado)
}
```

3. Utilize alguma função `readr::read_***_chunked` para importar o arquivo `flights.csv.zip`.

- Configure o tamanho do lote (`chunk`) para 100 mil registros;
- Configure a função de callback para instanciar DataFrames aplicando a função `getStats` criada acima;
- Configure o argumento `col_types` de forma que ele leia, diretamente do arquivo, apenas as colunas de interesse (veja nota de aula para identificar como realizar esta tarefa);

```
library(readr)

#### Criar minhas colunas de interesse
minhas_colunas = cols_only(
  YEAR = col_integer(),
  MONTH = col_integer(),
  DAY = col_integer(),
  AIRLINE = col_character(),
  ARRIVAL_DELAY = col_double())

in2 <- read_csv_chunked('flights.csv',
  callback = DataFrameCallback$new(getStats),
  chunk_size = 100000,
  col_types = minhas_colunas)

in2
```

```
# A tibble: 1,478 x 5
  MONTH DAY AIRLINE voos_atrasados total_voos
  <int> <int> <chr>          <int>      <int>
1     1     1 1 AA              514        1379
2     1     1 1 DL              132        1558
3     1     1 1 UA              367        1269
4     1     1 1 US              201        1028
5     1     2 2 AA              649        1508
6     1     2 2 DL              406        2261
7     1     2 2 UA              493        1411
8     1     2 2 US              229        1174
9     1     3 3 AA              767        1425
10    1     3 3 DL              702        2021
# i 1,468 more rows
```

4. Crie uma função chamada `computeStats` que:

a- Combine as estatísticas suficientes para compor a métrica final de interesse (percentual de atraso por dia/mês/cia aérea); b- Retorne as informações em um tibble contendo apenas as seguintes colunas: i. Cia: sigla da companhia aérea; ii. Data: data, no formato AAAA-MM-DD

(dica: utilize o comando `as.Date`); iii. Perc: percentual de atraso para aquela cia. aérea e data, apresentado como um número real no intervalo $[0,1]$.

```
computeStats = function(stats){
  resultado <- stats |>
  group_by(AIRLINE, MONTH, DAY) |>
  summarise(
    soma_atrasados = sum(voos_atrasados),
    soma_total = sum(total_voos),
    .groups = "drop"
  ) |>
  mutate(
    Cia = AIRLINE,
    Data = as.Date(paste("2015", MONTH, DAY, sep = "-")),
    Perc = soma_atrasados / soma_total
  ) |>
  select(Cia, Data, Perc)

  return(resultado)
}

resultado_final <- computeStats(in2)
```

5. Produza um mapa de calor em formato de calendário para cada Cia. Aérea.

- a. Instale e carregue os pacotes `ggcal` e `ggplot2`.
- b. Defina uma paleta de cores em modo gradiente. Utilize o comando `scale_fill_gradient`. A cor inicial da paleta deve ser `#4575b4` e a cor final, `#d73027`. A paleta deve ser armazenada no objeto `pal`.
- c. Crie uma função chamada `baseCalendario` que recebe 2 argumentos a seguir: `stats` (tibble com resultados calculados na questão 4) e `cia` (sigla da Cia. Aérea de interesse). A função deverá:
 - d. Criar um subconjunto de `stats` de forma a conter informações de atraso e data apenas da Cia. Aérea dada por `cia`.
- ii. Para o subconjunto acima, montar a base do calendário, utilizando `ggcal(x, y)`. Nesta notação, `x` representa as datas de interesse e `y`, os percentuais de atraso para as datas descritas em `x`.
- iii. Retornar para o usuário a base do calendário criada acima.
 - d. Executar a função `baseCalendario` para cada uma das Cias. Aéreas e armazenar os resultados, respectivamente, nas variáveis: `cAA`, `cDL`, `cUA` e `cUS`.

- e. Para cada uma das Cias. Aéreas, apresente o mapa de calor respectivo utilizando a combinação de camadas do ggplot2. Lembre-se de adicionar um título utilizando o comando ggtitle. Por exemplo, cXX + pal + ggtitle("Título").

```
library(ggplot2)
library(ggcal)

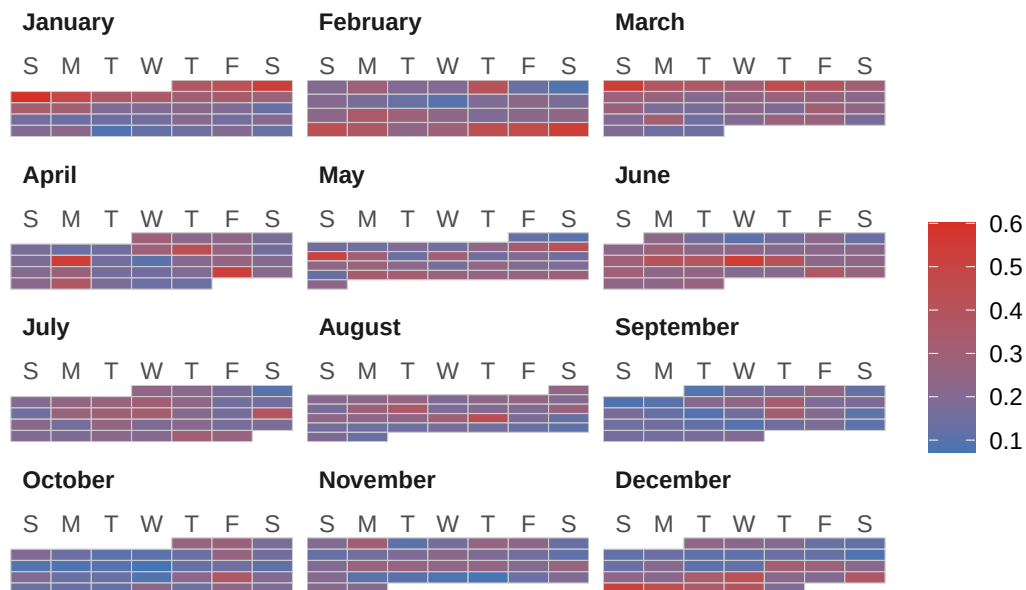
pal <- scale_fill_gradient(low = "#4575b4", high = "#d73027", name = "Atraso em %")

# Corrigido o nome da função (baseCalendario) e os parâmetros do ggcal (dates e fills)
baseCalendario <- function(stats, cia) {
  stats_cia <- stats |> filter(Cia == cia)
  calendario_base <- ggcal(dates = stats_cia$Data, fills = stats_cia$Perc)
  return(calendario_base)
}

cAA <- baseCalendario(resultado_final, "AA")
cDL <- baseCalendario(resultado_final, "DL")
cUA <- baseCalendario(resultado_final, "UA")
cUS <- baseCalendario(resultado_final, "US")

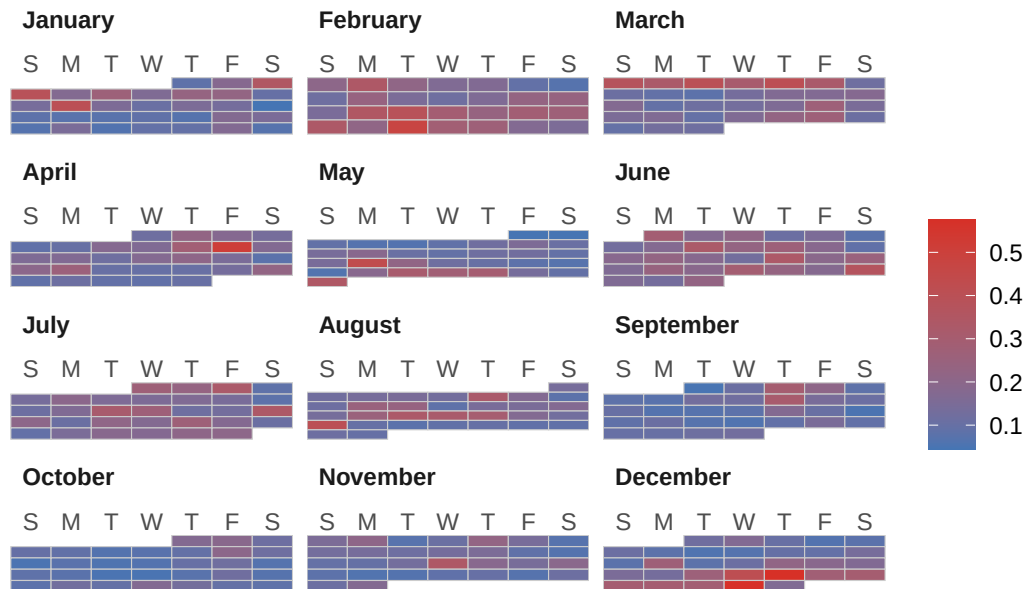
cAA + pal + ggtitle("Percentual de Atrasos - American Airlines (AA)")
```

Percentual de Atrasos – American Airlines (AA)



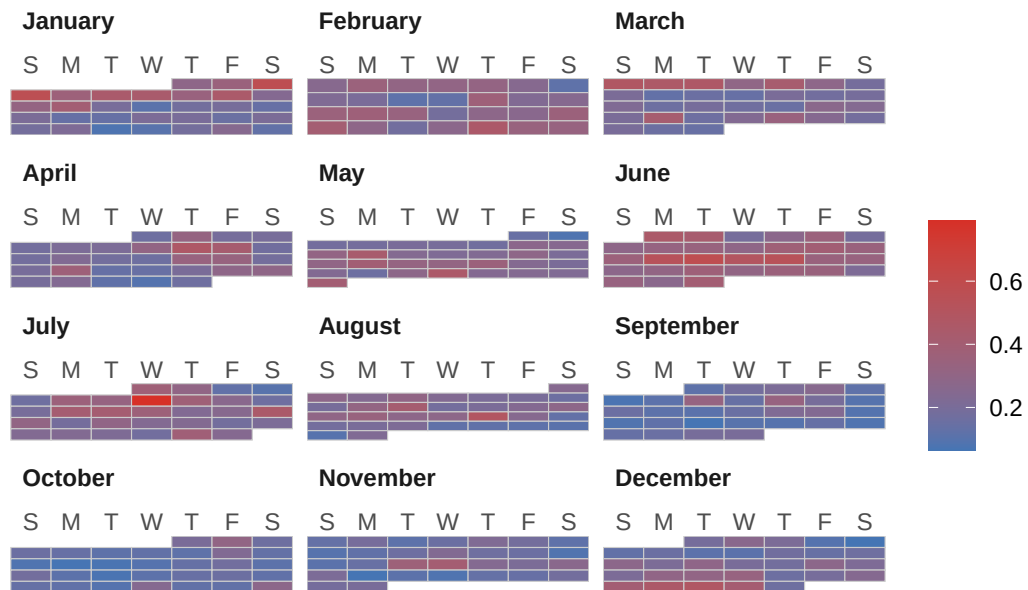
```
cDL + pal + ggtitle("Percentual de Atrasos - Delta Air Lines (DL)")
```

Percentual de Atrasos – Delta Air Lines (DL)



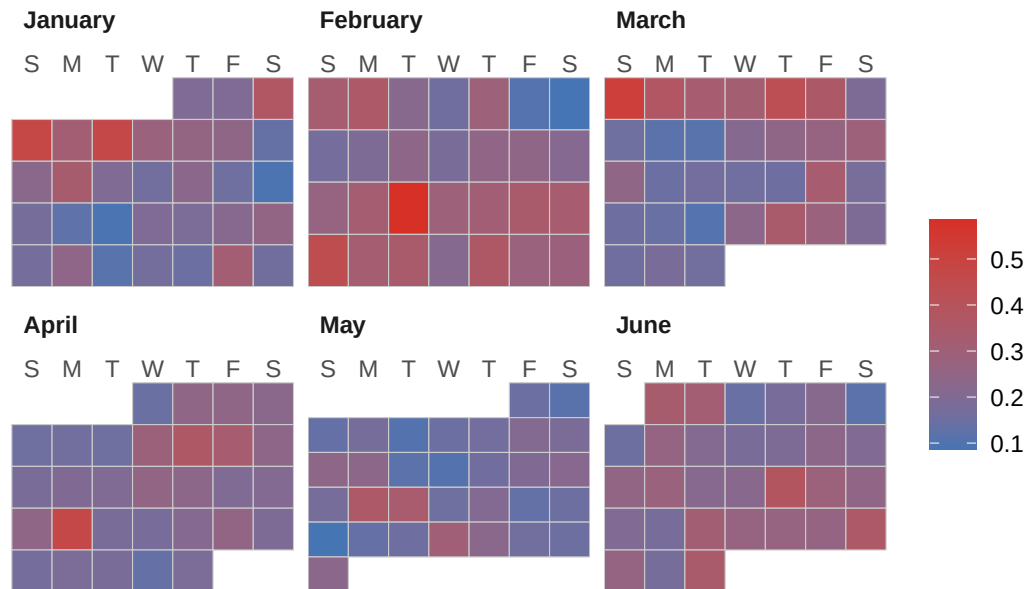
```
cUA + pal + ggtitle("Percentual de Atrasos - United Airlines (UA)")
```

Percentual de Atrasos – United Airlines (UA)



```
cUS + pal + ggtitle("Percentual de Atrasos - US Airways (US)")
```

Percentual de Atrasos – US Airways (US)



```
Sys.setenv(TV = "America/Sao_Paulo")
Sys.time()
```

```
[1] "2026-08-25 13:20:05 UTC"
```